ARCHITECTURE REVIEW · PART II · WORKFLOW & STATE MACHINE DESIGN

State Machine–Driven *UI* Workflow Architecture

Observations on window/form lifecycle, command-driven graph navigation, execution context binding, and UI derivation from workflow definitions

---

OBSERVATION 01

What You Are Describing Has a Formal Name

The architecture you have outlined independently converges on a well-established formal pattern: a **Hierarchical Finite State Machine (HFSM)** applied to application workflow. This is not a new idea — its theoretical foundations were published by David Harel in 1987 as *Statecharts*, and it underlies modern workflow engines, UI frameworks such as XState, and the navigation architecture of many clinical information systems.

The correspondence is direct and exact:

- **Nodes** — application states; each corresponds to an active window or form
- **Edges** — transitions triggered by commands; some mandatory (ordered), some optional (free-roaming)
- **Node data** — the execution context accumulated and contributed at each state
- **Paths through the graph** — workflows; ordered paths enforce clinical sequences, optional paths support flexible navigation

> **Validation.** Arriving at this pattern from first principles — driven by your architecture's constraints rather than by framework selection — is a strong signal that the design is well-reasoned. The

> constraints of VistA, the need for multiple client types, and the evolving domain model all point naturally toward state machine–driven workflow.

OBSERVATION 02

Every Node Is Associated with an Execution Context

Your intuition is correct — and the relationship is even richer than association. Each node in the graph **declares a three-part context contract**: what context it requires to be entered, what context it contributes upon exit, and what context each outgoing transition requires to be enabled. The execution context is therefore not merely associated with nodes — it is the **connective tissue of the entire graph**.

Node Context Contract — Structural Definition

```
Node: SelectPatient
  entry_requires:   []                              ← always reachable
  exit_contributes: [PatientRecord.dfn,
                     PatientRecord.name]
  transitions:
    → SelectTemplate   requires: [PatientRecord.dfn]   via: cmd.selectPatient
    → Cancel           requires: []                    via: cmd.cancel

Node: SelectTemplate
  entry_requires:   [PatientRecord.dfn]
  exit_contributes: [Template.ien, Template.name]
  transitions:
    → DocumentVisit    requires: [PatientRecord.dfn,
                                  Template.ien]          via: cmd.selectTemplate
    → SelectPatient    requires: []                    via: cmd.back

Node: DocumentVisit
  entry_requires:   [PatientRecord.dfn, Template.ien]
  exit_contributes: [Document.ien, Document.status]
  transitions:
    → SignDocument     requires: [Document.ien]          via: cmd.sign
    → SaveDraft        requires: [Document.ien]          via: cmd.saveDraft
```

This context-contract model means that **transition availability is fully computable at runtime** without any hand-coded enable/disable logic in the UI. The state machine engine evaluates whether the current context satisfies each transition's requirements and publishes the result. The UI simply observes and renders.

OBSERVATION 03

The Workflow Graph Is Completely Independent of UI

This is one of the most consequential properties of the architecture. The workflow graph is a **pure domain artifact**. It describes what the application does and in what order, with no reference to pixels, layouts, widgets, or rendering technology.

- Stored as **data** — JSON, XML, a domain-specific language, or a graph database
- **Validated and tested** without instantiating any UI component
- Executed by a **state machine engine** that has no rendering dependency
- **Shared identically** across the Windows GUI client, ChUI console client, and any future web client

The UI becomes a **renderer** of the current state machine state. Different clients render the same state differently — a Windows form, a terminal menu, a web page — but the workflow logic is identical across all of them. The state machine engine is written once.

> **Clinical significance.** In healthcare, workflow correctness is a patient safety concern, not merely a UX preference. A formal, UI-independent graph is reviewable by clinical informaticists and governance bodies in a way that UI mockups or procedural code are not. This separation makes formal clinical review tractable.

OBSERVATION 04

The Graph Can and Should Be Designed Before the UI

Because the workflow graph is a pure domain artifact, it can be authored, reviewed, and validated entirely before any UI implementation begins. This is a **significant practical and clinical advantage**.

WHAT BECOMES POSSIBLE BEFORE ANY UI EXISTS

- **Stakeholder review as diagrams** — clinicians, informaticists, and domain experts can review and approve workflows as state diagrams, without needing a UI prototype.
- **Early gap detection** — missing transitions, unreachable states, deadlocks, and ambiguous branching all surface as structural graph problems before any code is written.
- **Shared contract for parallel development** — developers implementing different modules work from the same unambiguous workflow definition simultaneously.
- **Automated workflow testing** — the state machine engine can be driven by scripted context inputs and command sequences, validating clinical logic with no UI dependency.
- **Requirements traceability** — each node and transition is a named, addressable requirement. Change requests map to specific graph elements rather than to scattered UI code.

OBSERVATION 05

What Can — and Cannot — Be Derived from the Graph

The answer to whether the UI can be generated from the workflow graph is **precisely partial**. The graph provides the skeleton and nervous system of the UI. What requires deliberate design is the skin and muscle — the visual, communicative, and accessibility layer that no structural definition can supply.

DERIVABLE FROM THE GRAPH	REQUIRES DELIBERATE DESIGN
✓ Navigation controls — Back, Next, Cancel — and their enabled/disabled state	✗ Layout and visual composition — the graph declares context, not presentation
✓ Command availability per state, driven by context requirements	✗ Information density — what to show, summarize, or hide in each state
✓ Workflow progress indicators — breadcrumb, step counter, progress bar	✗ Clinician-facing error and edge-case language

- ✓ Validation gate behavior — transitions blocked until context requirements are met
- ✓ Inline validation messages — which fields or objects are missing for a given transition
- ✓ Screen titles and workflow step labels
- ✓ Reachability analysis — which states are accessible from the current state

- ✗ Accessibility patterns — tab order, keyboard navigation, screen reader semantics
- ✗ Visual hierarchy and emphasis within a form
- ✗ Interaction patterns specific to the client platform

> **Design implication.** A practical strategy is to auto-generate a *functional scaffold* — a working but unstyled UI — directly from the graph. This scaffold is fully navigable and command-operable from day one. Visual design is applied progressively on top of the scaffold. The scaffold and the visual layer are separated by a clean interface.

OBSERVATION 06

Two Transition Types Must Be Modeled Explicitly

Since some workflows require strict ordering and others allow flexible navigation, the graph definition must distinguish transition types explicitly — not enforce this distinction implicitly in UI code. Keeping it in the graph ensures the logic remains in the UI-independent layer and remains auditable.

TYPE	BEHAVIOR	CONTEXT ENFORCEMENT	EXAMPLE	TAG
`Mandatory / Sequential`	Must follow specified path; no bypass permitted	Hard — transition blocked if context unsatisfied	Patient selection must precede template selection	MANDATORY
`Optional / Reentrant`	User may navigate freely	Soft — available if context	Switching between open	OPTIONAL

TYPE	BEHAVIOR	CONTEXT ENFORCEMENT	EXAMPLE	TAG
	among permitted states	requirements met	sections of a visit note	
Conditional	Path depends on context value, not just presence	Guard expression evaluated at transition time	Inpatient vs outpatient template sets	CONDITIONAL
Global / Interrupt	Available from any state regardless of current node	Independent of local context; app-level command	Emergency patient lookup; logout	GLOBAL

Global transitions deserve particular attention in a healthcare context. Clinical emergencies require the ability to interrupt any workflow state and reach a target state immediately. Modeling this explicitly — rather than as a special-case UI hack — ensures the behavior is consistent, testable, and auditable across all client types.

OBSERVATION 07

The Complete Layered Architecture

Incorporating the workflow state machine, the full architecture now comprises five layers. Three of the five are entirely UI-independent — meaning the vast majority of application logic can be developed, tested, and validated without any UI implementation.

5	UI Shell — Windows GUI · ChUI Console · Web	UI-DEPENDENT

	Renders current state machine state · Dispatches commands · Platform-specific · Replaceable	
4	**Workflow State Machine Engine** Graph of states · Transition rules · Context contracts · Mandatory vs optional paths	UI-INDEPENDENT
3	**Command Registry + Execution Context** Self-describing commands · Context validation · Context resolution · Declarative param mapping	UI-INDEPENDENT
2	**RPC Gateway** HTTP / JSON · VistA RPC registry · Error normalization · Session & auth boundary	UI-INDEPENDENT
1	**VistA / FileMan / Mumps** Persistence · Clinical data · Existing RPC surface · Session globals	EXTERNAL

Three of five layers are UI-independent. Automated testing covers the vast majority of application logic — workflow correctness, command execution, context management, and RPC integration — without requiring any UI component. Multiple client types share layers 1–4 entirely.

⚠ PRIMARY ARCHITECTURAL RISK — CONTEXT SCHEMA GOVERNANCE

As the healthcare domain model evolves, the execution context object graph will grow in complexity. The most common failure mode for this pattern is the context becoming an **untyped property bag** that different modules and workflow nodes interpret inconsistently.

Design a formal **Context Schema** from the outset — even before the domain model is finalized. The shape of how objects enter, mutate, resolve, and leave context must be governed centrally. In a healthcare system where data integrity is a regulatory and patient safety requirement, undisciplined context evolution is not recoverable without significant rework.

Healthcare Application Architecture · Part II

Workflow State Machine Design · VistA / Mumps